

Параллелизмы в верификации нейронных сетей

Дипломная работа

Воробьев Сергей Юрьевич

Научный руководитель:

Ильюшин Евгений Альбинович

Московский государственный университет имени М. В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра информационной безопасности

7 апреля 2026 г.



Содержание

Задача верификации нейронных сетей

Распараллеливание верификации

Выбор метода параллелизма

Реализация

Эксперименты

Заключение



Формальная верификация нейронных сетей

Задача

Доказать, что для любого входа $x \in C$ (область возмущений), выход сети $f(x)$ принадлежит безопасному множеству S .

Пример: Верификация устойчивости к состязательным атакам

- ▶ Дано: изображение x , модель f , радиус возмущения ε
- ▶ Проверить: для всех $x' : \|x - x'\|_\infty \leq \varepsilon$ выполняется $\arg \max f(x) = \arg \max f(x')$



Методы верификации: CROWN

Алгоритм CROWN

Находит линейные границы выхода сети через обратное распространение:

$$\underline{W}_j x + \underline{b}_j \leq f_j(x) \leq \overline{W}_j x + \overline{b}_j$$

Ключевая операция: $A_{prev} = A_{curr} \cdot W$

- ▶ A — матрица символьных границ, размер:
($Batch \times Spec \times Neurons$)
- ▶ Для глубоких сетей матрицы A занимают огромный объем памяти
- ▶ **AlphaBetaCROWN** — state-of-the-art верификатор
(победитель VNN-COMP 2021-2023)



Методы верификации: IBP

Interval Bound Propagation (IBP)

Послойное распространение интервалов — самый быстрый, но наименее точный метод

Формулы для линейного слоя с W и входным интервалом $[\ell, u]$:

$$\ell_{out} = W^+ \ell + W^- u + b, \quad u_{out} = W^+ u + W^- \ell + b$$

где $W^+ = \max(W, 0)$, $W^- = \min(W, 0)$

Сравнение IBP и CROWN:

- ▶ **CROWN** — сохраняет линейную зависимость $y = Ax + b$ через все слои, учитывает корреляции между переменными
- ▶ **IBP** — каждый слой рассматривает входы как независимые $\Rightarrow$ *wrapping effect*: ширина bounds растёт экспоненциально с глубиной
- ▶ IBP: $O(d)$ по времени, минимум памяти; CROWN: $O(d^2)$, но значительно точнее



Проблема масштабируемости

Барьер памяти (Memory Wall)

Размер матриц A растет как $O(\text{Batch} \times \text{Spec} \times \text{Neurons})$

Пример: Для трансформера с $N = 4096$, $B = 128$, $S = 10$:

- ▶ Одна матрица A : ~ 20 МБ
- ▶ Для всех слоев: может превысить 80 ГБ на A100

Разрыв возможностей

- ▶ **Обучение:** модели на миллиарды параметров, тысячи GPU
- ▶ **Верификация:** ограничена памятью одного GPU



Текущее состояние: Data Parallelism

AlphaBetaCROWN использует параллелизм по данным

Алгоритм Branch-and-Bound разбивает входное пространство на подзадачи, каждая решается на отдельном GPU

Ограничение:

- ▶ Решает проблему **времени вычислений**
- ▶ НЕ решает проблему **памяти**
- ▶ Если одна задача не влезает в память одного GPU — верификация невозможна

Необходимость модельного параллелизма

Нужно разделить саму модель и матрицы A между GPU



Опыт из обучения моделей

Три основных подхода к распределенному обучению:

1. Tensor Parallelism (TP)

- ▶ Разделение матриц внутри слоя
- ▶ Megatron-LM, широко используется для LLM

2. Pipeline Parallelism (PP)

- ▶ Разделение слоев модели по глубине
- ▶ GPipe, используется для очень глубоких сетей

3. Fully Sharded Data Parallelism (FSDP)

- ▶ Шардирование параметров и оптимизатора
- ▶ FSDP (PyTorch), ZeRO (DeepSpeed)

Вопрос: Какой подход применим к верификации?



Математика обратного распространения границ

Ключевая операция CROWN

$$A_{prev} = A_{curr} \cdot W$$

где A — матрица символьных границ, W — веса слоя

Для линейного слоя:

- ▶ A_{curr} : размер $(B \times S \times N_{out})$
- ▶ W : размер $(N_{out} \times N_{in})$
- ▶ A_{prev} : размер $(B \times S \times N_{in})$

Аналогия с обучением

Это похоже на обратное распространение, но вместо градиентов мы передаем **символьные матрицы границ**



Сравнение подходов для верификации

Характеристика	TP	PP	FSDP
Шардирует веса	Да	Да	Да
Шардирует A -матрицы	Да	Нет	Нет
Точность bounds	Падает с глубиной	—	Побитово идентична
Коммуникация	AllReduce/слой	Point-to-point	AllGather/слой
Совместимость с β -CROWN	Сложно	Невозможно	Да

Вывод: TP и FSDP — два полюса компромисса *память–точность*



Tensor Parallelism: Column и Row Parallel

Column Parallel — веса разделены по **выходной** размерности:

- ▶ $W = [W_0 | W_1]$ разделен между GPU
- ▶ Обратный CROWN: A_{curr} уже разделена $\Rightarrow$ локально
 $P_i = A_i \cdot W_i$
- ▶ **AllReduce**: $A_{prev} = P_0 + P_1$

Row Parallel — веса разделены по **входной** размерности:

- ▶ Локально: $A_{prev,i} = A_{curr} \cdot W_i$
- ▶ Результат автоматически разделен, **коммуникация не нужна**

Пара Col-Row = шардированная зона

Между Col и Row активации имеют размерность h/P на каждом GPU



Проблема ТР: потеря точности bounds

Фундаментальное ограничение

Внутри шардированных зон CROWN backward невозможен без полных весов $\Rightarrow$ используется IBP (Interval Bound Propagation)

IBP vs CROWN:

- ▶ **CROWN** — глобальная линейная зависимость $y = Ax + b$, учитывает корреляции
- ▶ **IBP** — послойные интервалы, теряет корреляции (*wrapping effect*)
- ▶ Ширина IBP-bounds растёт экспоненциально с числом слоёв

Следствие: ТР эффективен для 1–2 шардированных зон; для глубоких моделей bounds становятся бесполезными



FSDP для верификации

Идея

Шардировать *только веса*, собирая полную матрицу AllGather перед каждым слоем и освобождая после

Преимущества:

- ▶ Bounds **побитово идентичны** single-GPU (каждый слой получает полную W)
- ▶ Не изменяет вычислительный граф $\Rightarrow$ совместим с β -CROWN
- ▶ Baseline-память (веса) сокращается на 80–90%

Компромисс:

- ▶ Пиковая память снижается на 34–39% (A-матрицы полноразмерные)
- ▶ TP даёт $\sim 2\times$ снижение пиковой памяти, но теряет точность



Почему не Pipeline Parallelism?

PP несовместим с верификацией

1. CROWN backward требует весов *всех* слоёв одновременно
2. При PP каждый GPU хранит только свою стадию $\Rightarrow$ пересылка весов сводит PP к менее эффективному варианту TP
3. НЕ шардирует матрицы A — проблема памяти остаётся



Архитектура auto_LiRPA

auto_LiRPA

Библиотека для автоматического вычисления границ (Linear Relaxation based Perturbation Analysis)

Ключевые компоненты:

- ▶ BoundedModule — обёртка над PyTorch/ONNX моделью
- ▶ Bound-операторы для каждого типа слоя
- ▶ Методы bound_backward (CROWN) и IBP_general (IBP)



Реализация TP: операторы и шардирование

Компоненты TP (~500 строк):

1. BoundLinearTP_Col / BoundLinearTP_Row
 - ▶ Наследуют от BoundLinear, добавляют AllReduce в backward
 - ▶ ONNX-интеграция через torch.autograd.Function
2. tp_shard_bounded_module
 - ▶ Автоматическое шардирование произвольного BoundedModule
 - ▶ Замена чередующихся пар BoundLinear на Col/Row
 - ▶ Зонирование подграфов для промежуточных bounds



Реализация FSDP: послойный AllGather

Компоненты FSDP (~100 строк):

1. `fsdp_shard_bounded_module`
 - ▶ Шардирование BoundParams (веса): $W \rightarrow W^{(r)} \in \mathbb{R}^{(k/P) \times n}$
2. Прозрачный AllGather в `BoundParams.forward()`
 - ▶ Полная матрица собирается и возвращается как результат `forward`
3. Хуки `fsdp_gather_node` / `fsdp_free_node`
 - ▶ Интегрированы в CROWN backward и IBP forward
 - ▶ В каждый момент в памяти ≤ 1 полной матрицы (помимо шардов)

Ключевое отличие от TP

Не создаются новые операторы, не изменяется вычислительный граф, не вводятся шардированные зоны



Статус реализации

Tensor Parallelism

- ✓ Операторы BoundLinearTP_Col/Row с ONNX-интеграцией
- ✓ Автоматическое шардирование (PyTorch и ONNX модели)
- ✓ Зонирование подграфов, IBP fallback
- ✓ Эксперименты: память, корректность, VNN-COMP

FSDP

- ✓ Шардирование весов с послойным AllGather/Free
- ✓ Поддержка IBP и CROWN
- ✓ Побитово идентичные bounds для всех моделей
- ✓ Эксперименты: корректность, память



Аппаратная конфигурация и план

Аппаратура: 2× NVIDIA A40 (48 ГБ), NVLink, NCCL, PyTorch 2.x

5 групп экспериментов:

1. TP: потребление GPU-памяти (OOM-тест)
2. TP: корректность и soundness bounds
3. TP: автоматическое шардирование моделей VNN-COMP
4. FSDP: корректность bounds
5. FSDP: потребление GPU-памяти



Эксперимент 1: TP — потребление памяти

Модель: MLP, $D=4096$, $H=262144$, batch=2048, CROWN, $\varepsilon=0.01$

Режим	GPU	max_allocated, МБ	max_reserved, МБ
Single GPU	1×A40	26 826	30 874
TP = 2	GPU 0	13 513	15 512
TP = 2	GPU 1	13 513	15 512

Результат

Коэффициент снижения: $26826/13513 \approx \mathbf{1.985\times}$ (теор. предел 2.0×)



Эксперимент 2: TP — корректность bounds

Метод: CROWN, $\epsilon=0.02$; сравнение TP=2 с single-GPU эталоном

Модель	Пар Col-Row	$ lb_{\text{diff}} $	$ ub_{\text{diff}} $
PyTorch 256×2	1	$3 \cdot 10^{-8}$	$3 \cdot 10^{-8}$
PyTorch 256×4	2	$2.5 \cdot 10^0$	$2.5 \cdot 10^0$
ONNX 256×2	1	$6 \cdot 10^{-8}$	$6 \cdot 10^{-8}$
ONNX 256×4	2	$5.6 \cdot 10^0$	$4.0 \cdot 10^0$
ONNX 256×6	3	$7.9 \cdot 10^2$	$7.5 \cdot 10^2$

Вывод

Soundness подтверждена для всех конфигураций ($lb_{\text{tp}} \leq lb_{\text{ref}}$, $ub_{\text{tp}} \geq ub_{\text{ref}}$).

- ▶ 1 пара Col-Row: совпадение с точностью $\sim 10^{-8}$ (машинный ϵ)
- ▶ 2+ пар: bounds шире эталона, ошибка растёт на порядки (IBP fallback)



Эксперимент 3: автошардирование VNN-COMP моделей

Задача

Проверить `tr_shard_bounded_module` на ONNX-моделях MNIST-FC из VNN-COMP 2022

Процесс:

1. Загрузка ONNX $\rightarrow$ PyTorch (onnx2pytorch)
2. Обёртка в BoundedModule
3. Автоматическое шардирование без ручного вмешательства

Результат

Все 3 модели (256×2 , 256×4 , 256×6) успешно шардированы и прошли тест на soundness



Эксперимент 4: FSDP — корректность bounds

Метод: IBP + CROWN, FSDP=2 vs single-GPU; 8 конфигураций

Модель	Метод	$ \Delta lb $	$ \Delta ub $
MLP 256×2	IBP	0.00	0.00
MLP 256×2	CROWN	0.00	0.00
MLP 256×4	IBP	0.00	0.00
MLP 256×4	CROWN	0.00	0.00
MLP 256×6	CROWN	0.00	0.00
ONNX 256×2	CROWN	0.00	0.00
ONNX 256×4	CROWN	0.00	0.00

Результат

Побитово идентичные bounds для всех моделей и методов ($\Delta = 0.00$, не $\sim 10^{-8}$). Кросс-ранговое расхождение: 0.00.



Эксперимент 5: FSDP — потребление памяти

Baseline-память (веса):

Модель	Single	FSDP	↓%
$h=256, d=4$	26	11	57
$h=1024, d=4$	187	40	79
$h=4096, d=4$	2272	417	82
$h=8192, d=4$	8760	1594	82
$h=4096, d=8$	9721	930	90

Пиковая память:

Модель	Single	FSDP	↓%
$h=4096, d=4$	3800	2527	34
$h=8192, d=4$	14759	9782	34
$h=4096, d=8$	16726	10232	39

Размеры в МБ. Теория: $M_W^{\text{FSDP}} / M_W^{\text{single}} = 1/P + 1/d$



Сравнение TP и FSDP

	TP	FSDP
Пиковая память	~ 2× снижение	34–39% снижение
Baseline-память	~ 2× снижение	80–90% снижение
Точность bounds	Падает с глубиной	Побитово идентична
Совместимость с β -CROWN	Сложно	Да
Объём реализации	~500 строк	~100 строк

Вывод

TP — максимальная экономия памяти для неглубоких моделей;
FSDP — идеальная точность и совместимость с полной верификацией



Результаты работы

1. **Tensor Parallelism:** линейное снижение пиковой памяти ($\approx 2\times$ при $P=2$), но потеря точности bounds для глубоких моделей (IBP fallback)
2. **FSDP:** побитово идентичные bounds, экономия baseline-памяти 80–90%, пиковой 34–39%
3. **Автоматическое шардирование:** обе функции работают с произвольными BoundedModule, включая ONNX-модели VNN-COMP
4. **Теоретический анализ:** формальная модель $M_{\text{peak}}^{\text{FSDP}} = h^2(d/P + 1 + C)$ подтверждена экспериментально



Направления дальнейшей работы

Полная верификация через FSDP

FSDP не изменяет вычислительный граф $\Rightarrow$ совместим с β -CROWN и Branch-and-Bound. Требуется распараллеливание управляющей логики BaB.

Расширения

- ▶ Поддержка свёрточных слоёв (BoundConv)
- ▶ Масштабирование на $P = 4, 8$ GPU
- ▶ Оптимизация α -CROWN с TP и FSDP



Спасибо за внимание!

